

Information-based Learning

Sections 4.1, 4.2, 4.3

Dr. Mohammed Brahimi and Dr. Aicha Boutorh



1 Big Idea

2 Fundamentals

- Decision Trees
- Shannon's Entropy Model
- Information Gain

3 Standard Approach: The ID3 Algorithm

- A Worked Example: Predicting Vegetation Distributions
- Alternative Decision Tree Algorithms: C4.5 and CART

4 Summary

Lecture Goals

- Introducing a predictive model focusing on **the most informative features**.
- Defines informative features as those dividing data into sets **homogeneous to the target feature**.
- Present **training algorithms** for the **decision trees**.

Big Idea

Guess-Who Game



(a) Brian



(b) John



(c) Aphra



(d) Aoife

Figure: Cards showing character faces and names for the *Guess-Who* game

Which question would you ask first:

- 1 Is it a man?
- 2 Does the person wear glasses?

Guess-Who Game

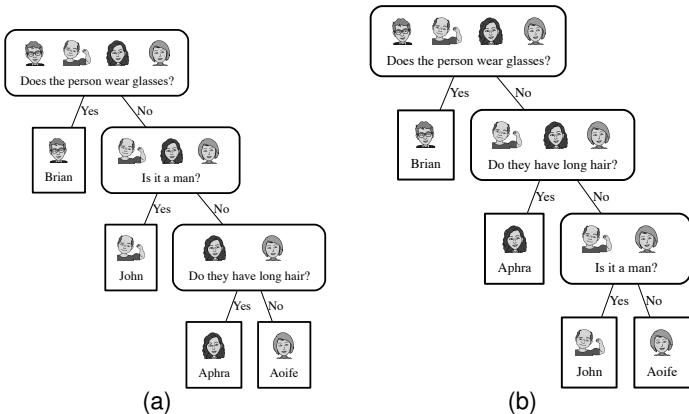
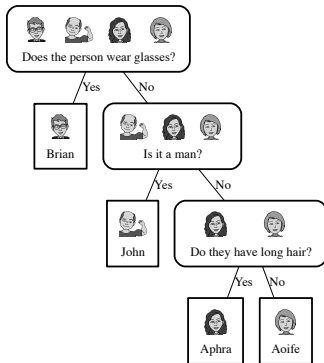


Figure: Different question sequences that can follow in a game of *Guess-Who* beginning with the question **Does the person wear glasses?**

Guess-Who Game

- In both of the diagrams:
 - One path is 1 question long,
 - One path is 2 questions long,
 - And two paths are 3 questions long.
- Consequently, if you ask Question (2) first, the average number of questions you have to ask per game is:

$$\frac{1 + 2 + 3 + 3}{4} = 2.25$$



Guess-Who Game

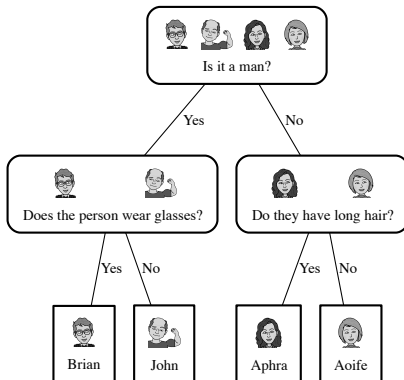
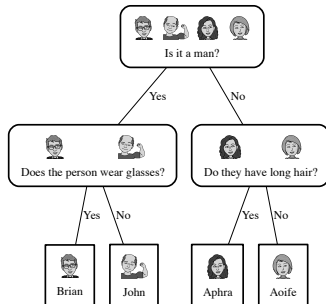


Figure: The different question sequences that can follow in a game of *Guess-Who* beginning with the question **Is it a man?**

Guess-Who Game

- All the paths in this diagram are two questions long.
- On average if you ask Question (1) first, the average number of questions you have to ask per game is:

$$\frac{2 + 2 + 2 + 2}{4} = 2$$



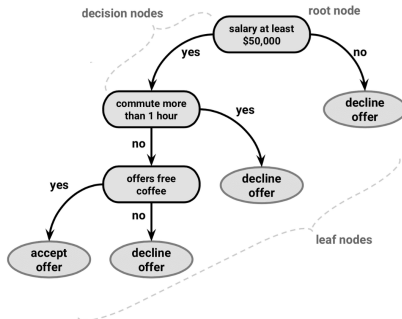
Guess-Who Game

- Question (1) yields more informative results than Question (2) on average, resulting in fewer follow-up questions.
- This difference is not solely based on the literal answers (YES or NO).
- It relates to how each question divides the domain into differently sized subsets, considering the likelihood of each possible answer.
 - Answer to Question 1 (man?) splits the domain into two sets of equal size → 1 more question enough
 - Answer to Question 2 (glasses?) splits the domain into two sets of 1 and 3 elements resp → up to 2 more questions may be needed.
 - Considering the likelihoods, Question 2 leaves one with more questions to ask on average.

Big Idea

- The central idea is to determine which features provide the most valuable information when asking questions. This involves considering two factors:
 - 1 how the domain is split up after the answer is received,
 - 2 The probability associated with each of the potential answers.

Fundamentals



- A decision tree consists of:
 - 1 a **root node** (or starting node),
 - 2 **interior nodes (or decision nodes)**,
 - 3 and **leaf nodes** (or terminating nodes).
- Non-leaf nodes (root and interior) test query's features, leaf node specifies a predicted class for the query.

Spam Prediction

Table: An email spam prediction dataset.

ID	SUSPICIOUS WORDS	UNKNOWN SENDER	CONTAINS IMAGES	CLASS
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

Decision Trees

	Suspicious	Unknown	Contains	
ID	Words	Sender	Images	Class
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

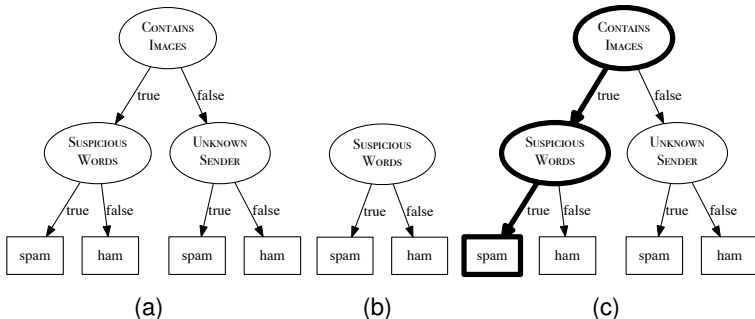
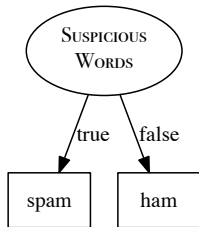
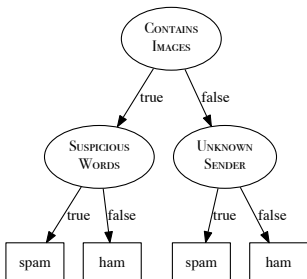


Figure: Trees (a) and (b) are consistent with the dataset. (c) shows the path to predict for query instance: SUSPICIOUS WORDS = 'true', UNKNOWN SENDER = 'true', CONTAINS IMAGES = 'true'.

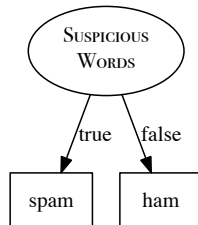
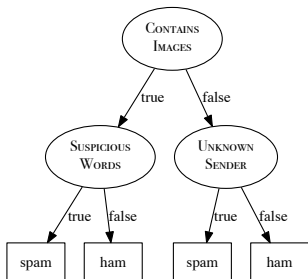
Spam Prediction



Important question

- Both of these trees will return identical predictions for all the examples in the training dataset.
- So, which tree should we use?

Spam Prediction



Occam's Razor

- Apply the same approach as we used in the game: prefer decision trees that use fewer tests (shallower trees).
- This is an example of Occam's Razor.

Spam Prediction

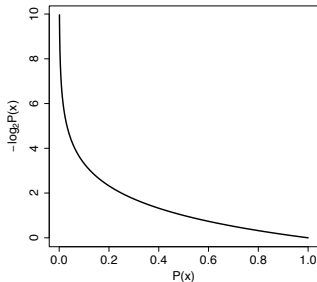
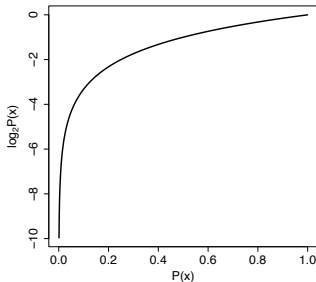
Creating Shallow Trees

- Testing SUSPICIOUS WORDS at the root creates a shallow tree by perfectly splitting data into '*spam*' and '*ham*'.
- Features that split the dataset into pure sets, provide information about the target feature.
- we can make shallow trees by testing the informative features early on in the tree.
- All we need to do that is a computational metric of the purity of a set: **entropy**

Shannon's Entropy Intuition

- Claude Shannon's entropy measures the randomness or impurity within a set.
- Entropy's essence lies in the **uncertainty** of predicting an outcome from a random selection in the set.
- Higher entropy means greater unpredictability, reflecting a diverse set where guessing the outcome is more challenging.
- Lower entropy, on the other hand, suggests less uncertainty, making it easier to anticipate the outcome.

Shannon's Entropy Intuition



- Entropy is related to the probability of an outcome.
 - High probability $\rightarrow$ Low entropy
 - Low probability $\rightarrow$ High entropy
- If we multiply **log** by -1 we get this mapping!

Shannon's Entropy

- Shannon's model of entropy is a weighted sum of the logs of the probabilities of each of the possible outcomes when we make a random selection from a set.

$$H(t) = - \sum_{i=1}^I (P(t = i) \times \log_s(P(t = i))) \quad (1)$$

Card Example

- What is the entropy of a set of 52 different playing cards?

$$\begin{aligned}
 H(card) &= - \sum_{i=1}^{52} P(card = i) \times \log_2(P(card = i)) \\
 &= - \sum_{i=1}^{52} \frac{1}{52} \times \log_2\left(\frac{1}{52}\right) \\
 &= -\log_2\left(\frac{1}{52}\right) \\
 &= 5.700 \text{ bits}
 \end{aligned}$$

Card Example

- What is the entropy of a set of 52 playing cards if we only distinguish between the cards based on their suit $\{\heartsuit, \clubsuit, \diamondsuit, \spadesuit\}$?
- $Pr(\heartsuit) = Pr(\clubsuit) = Pr(\diamondsuit) = Pr(\spadesuit) = \frac{13}{52}$.

Card Example

$$\begin{aligned}
 H(\text{suit}) &= - \sum_{l \in \{\heartsuit, \clubsuit, \diamondsuit, \spadesuit\}} P(\text{suit} = l) \times \log_2(P(\text{suit} = l)) \\
 &= -((P(\heartsuit) \times \log_2(P(\heartsuit))) + (P(\clubsuit) \times \log_2(P(\clubsuit))) \\
 &\quad + (P(\diamondsuit) \times \log_2(P(\diamondsuit))) + (P(\spadesuit) \times \log_2(P(\spadesuit)))) \\
 &= -\left(\left(\frac{13}{52} \times \log_2\left(\frac{13}{52}\right)\right) + \left(\frac{13}{52} \times \log_2\left(\frac{13}{52}\right)\right) \right. \\
 &\quad \left. + \left(\frac{13}{52} \times \log_2\left(\frac{13}{52}\right)\right) + \left(\frac{13}{52} \times \log_2\left(\frac{13}{52}\right)\right)\right) \\
 &= -((0.25 \times -2) + (0.25 \times -2) \\
 &\quad + (0.25 \times -2) + (0.25 \times -2)) \\
 &= 2 \text{ bits}
 \end{aligned}$$

Shannon's Entropy Model

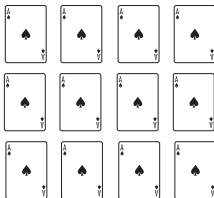
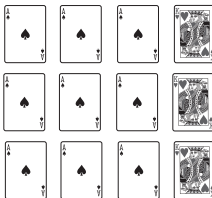
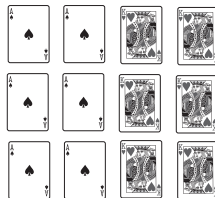
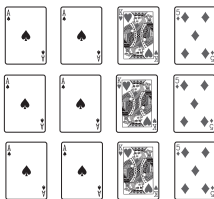
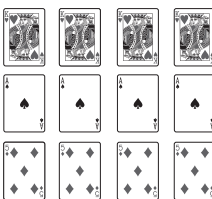
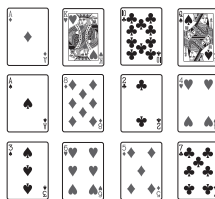
(g) $H(card) = 0.00$ (h) $H(card) = 0.81$ (i) $H(card) = 1.00$ (j) $H(card) = 1.50$ (k) $H(card) = 1.58$ (l) $H(card) = 3.58$

Figure: The entropy of different sets of playing cards measured in bits.

What is a good split?

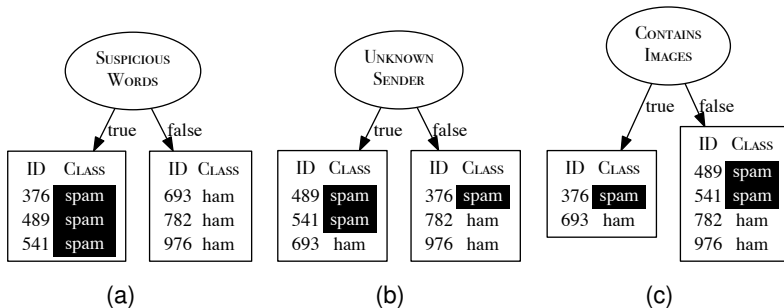


Figure: How the instances in the spam dataset split when we partition using each of the different descriptive features from the spam dataset in Table 1 ^[15]

Information Gain Intuition

- The intuition is that the ideal discriminatory feature will partition the data into **pure** subsets with instances having the same class.
 - SUSPICIOUS WORDS perfect split.
 - UNKNOWN SENDER mixture but some information (when *'true'* most instances are *'spam'*).
 - CONTAINS IMAGES no information.
- One way to implement this idea is to use a metric called **information gain**.

Information Gain

Information Gain

- The information gain of a descriptive feature can be understood as a measure of the reduction in the overall entropy of a prediction task by testing on that feature.

Information Gain Formula

Computing information gain involves the following 3 equations:

$$H(t, \mathcal{D}) = - \sum_{l \in \text{levels}(t)} (P(t = l) \times \log_2(P(t = l))) \quad (2)$$

$$\text{rem}(d, \mathcal{D}) = \sum_{l \in \text{levels}(d)} \underbrace{\frac{|\mathcal{D}_{d=l}|}{|\mathcal{D}|}}_{\text{weighting}} \times \underbrace{H(t, \mathcal{D}_{d=l})}_{\text{entropy of partition } \mathcal{D}_{d=l}} \quad (3)$$

$$IG(d, \mathcal{D}) = H(t, \mathcal{D}) - \text{rem}(d, \mathcal{D}) \quad (4)$$

Spam Prediction

- As an example we will calculate the information gain for each of the descriptive features in the spam email dataset.

Spam Prediction

- Calculate the **entropy** for the target feature in the dataset.

$$H(t, \mathcal{D}) = - \sum_{l \in \text{levels}(t)} (P(t = l) \times \log_2(P(t = l)))$$

ID	SUSPICIOUS WORDS	UNKNOWN SENDER	CONTAINS IMAGES	CLASS
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

Spam Prediction

$$\begin{aligned}
 H(t, \mathcal{D}) &= - \sum_{l \in \{ 'spam', 'ham' \}} (P(t = l) \times \log_2(P(t = l))) \\
 &= - ((P(t = 'spam') \times \log_2(P(t = 'spam'))) \\
 &\quad + (P(t = 'ham') \times \log_2(P(t = 'ham')))) \\
 &= - \left(\left(\frac{3}{6} \times \log_2\left(\frac{3}{6}\right) \right) + \left(\frac{3}{6} \times \log_2\left(\frac{3}{6}\right) \right) \right) \\
 &= 1 \text{ bit}
 \end{aligned}$$

Spam Prediction

- Calculate the **remainder** for the SUSPICIOUS WORDS feature in the dataset.

$$rem(d, \mathcal{D}) = \sum_{l \in levels(d)} \underbrace{\frac{|\mathcal{D}_{d=l}|}{|\mathcal{D}|}}_{\text{weighting}} \times \underbrace{H(t, \mathcal{D}_{d=l})}_{\text{entropy of partition } \mathcal{D}_{d=l}}$$

ID	SUSPICIOUS WORDS	UNKNOWN SENDER	CONTAINS IMAGES	CLASS
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

Spam Prediction

$rem(WORDS, \mathcal{D})$

$$\begin{aligned}
 &= \left(\frac{|\mathcal{D}_{WORDS=T}|}{|\mathcal{D}|} \times H(t, \mathcal{D}_{WORDS=T}) \right) + \left(\frac{|\mathcal{D}_{WORDS=F}|}{|\mathcal{D}|} \times H(t, \mathcal{D}_{WORDS=F}) \right) \\
 &= \left(\frac{3}{6} \times \left(- \sum_{l \in \{ 'spam', 'ham' \}} P(t=l) \times \log_2(P(t=l)) \right) \right) \\
 &\quad + \left(\frac{3}{6} \times \left(- \sum_{l \in \{ 'spam', 'ham' \}} P(t=l) \times \log_2(P(t=l)) \right) \right) \\
 &= \left(\frac{3}{6} \times \left(- \left(\left(\frac{3}{3} \times \log_2\left(\frac{3}{3}\right) \right) + \left(\frac{0}{3} \times \log_2\left(\frac{0}{3}\right) \right) \right) \right) \right) \\
 &\quad + \left(\frac{3}{6} \times \left(- \left(\left(\frac{0}{3} \times \log_2\left(\frac{0}{3}\right) \right) + \left(\frac{3}{3} \times \log_2\left(\frac{3}{3}\right) \right) \right) \right) \right) = 0 \text{ bits}
 \end{aligned}$$

Spam Prediction

- Calculate the **remainder** for the UNKNOWN SENDER feature in the dataset.

$$rem(d, \mathcal{D}) = \sum_{l \in levels(d)} \underbrace{\frac{|\mathcal{D}_{d=l}|}{|\mathcal{D}|}}_{\text{weighting}} \times \underbrace{H(t, \mathcal{D}_{d=l})}_{\text{entropy of partition } \mathcal{D}_{d=l}}$$

ID	SUSPICIOUS WORDS	UNKNOWN SENDER	CONTAINS IMAGES	CLASS
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

Spam Prediction

$$\text{rem}(\text{SENDER}, \mathcal{D})$$

$$\begin{aligned}
 &= \left(\frac{|\mathcal{D}_{\text{SENDER}=T}|}{|\mathcal{D}|} \times H(t, \mathcal{D}_{\text{SENDER}=T}) \right) + \left(\frac{|\mathcal{D}_{\text{SENDER}=F}|}{|\mathcal{D}|} \times H(t, \mathcal{D}_{\text{SENDER}=F}) \right) \\
 &= \left(\frac{3}{6} \times \left(- \sum_{l \in \{\text{'spam'}, \text{'ham'}\}} P(t=l) \times \log_2(P(t=l)) \right) \right) \\
 &\quad + \left(\frac{3}{6} \times \left(- \sum_{l \in \{\text{'spam'}, \text{'ham'}\}} P(t=l) \times \log_2(P(t=l)) \right) \right) \\
 &= \left(\frac{3}{6} \times \left(- \left(\left(\frac{2}{3} \times \log_2\left(\frac{2}{3}\right) \right) + \left(\frac{1}{3} \times \log_2\left(\frac{1}{3}\right) \right) \right) \right) \right) \\
 &\quad + \left(\frac{3}{6} \times \left(- \left(\left(\frac{1}{3} \times \log_2\left(\frac{1}{3}\right) \right) + \left(\frac{2}{3} \times \log_2\left(\frac{2}{3}\right) \right) \right) \right) \right) = 0.9183 \text{ bits}
 \end{aligned}$$

Spam Prediction

- Calculate the **remainder** for the CONTAINS IMAGES feature in the dataset.

$$rem(d, \mathcal{D}) = \sum_{l \in levels(d)} \underbrace{\frac{|\mathcal{D}_{d=l}|}{|\mathcal{D}|}}_{\text{weighting}} \times \underbrace{H(t, \mathcal{D}_{d=l})}_{\text{entropy of partition } \mathcal{D}_{d=l}}$$

	SUSPICIOUS	UNKNOWN	CONTAINS	
ID	WORDS	SENDER	IMAGES	CLASS
376	true	false	true	spam
489	true	true	false	spam
541	true	true	false	spam
693	false	true	true	ham
782	false	false	false	ham
976	false	false	false	ham

Spam Prediction

$$\begin{aligned}
 & \text{rem}(\text{IMAGES}, \mathcal{D}) \\
 &= \left(\frac{|\mathcal{D}_{\text{IMAGES}=T}|}{|\mathcal{D}|} \times H(t, \mathcal{D}_{\text{IMAGES}=T}) \right) + \left(\frac{|\mathcal{D}_{\text{IMAGES}=F}|}{|\mathcal{D}|} \times H(t, \mathcal{D}_{\text{IMAGES}=F}) \right) \\
 &= \left(2/6 \times \left(- \sum_{l \in \{\text{'spam'}, \text{'ham'}\}} P(t=l) \times \log_2(P(t=l)) \right) \right) \\
 &+ \left(4/6 \times \left(- \sum_{l \in \{\text{'spam'}, \text{'ham'}\}} P(t=l) \times \log_2(P(t=l)) \right) \right) \\
 &= \left(2/6 \times \left(- \left(\left(1/2 \times \log_2(1/2) \right) + \left(1/2 \times \log_2(1/2) \right) \right) \right) \right) \\
 &+ \left(4/6 \times \left(- \left(\left(2/4 \times \log_2(2/4) \right) + \left(2/4 \times \log_2(2/4) \right) \right) \right) \right) = 1 \text{ bit}
 \end{aligned}$$

Spam Prediction

- Calculate the **information gain** for the three descriptive features in the dataset.

$$IG(d, \mathcal{D}) = H(t, \mathcal{D}) - \text{rem}(d, \mathcal{D})$$

Spam Prediction

$$\begin{aligned}
 IG(\text{SUSPICIOUS WORDS}, \mathcal{D}) &= H(\text{CLASS}, \mathcal{D}) - \text{rem}(\text{SUSPICIOUS WORDS}, \mathcal{D}) \\
 &= 1 - 0 = 1 \text{ bit}
 \end{aligned}$$

$$\begin{aligned}
 IG(\text{UNKNOWN SENDER}, \mathcal{D}) &= H(\text{CLASS}, \mathcal{D}) - \text{rem}(\text{UNKNOWN SENDER}, \mathcal{D}) \\
 &= 1 - 0.9183 = 0.0817 \text{ bits}
 \end{aligned}$$

$$\begin{aligned}
 IG(\text{CONTAINS IMAGES}, \mathcal{D}) &= H(\text{CLASS}, \mathcal{D}) - \text{rem}(\text{CONTAINS IMAGES}, \mathcal{D}) \\
 &= 1 - 1 = 0 \text{ bits}
 \end{aligned}$$

- The results of these calculations match our intuitions.

Standard Approach: The ID3 Algorithm

ID3 Intuition

- ID3 Algorithm (Iterative Dichotomizer 3)
- Attempts to create the shallowest tree that is consistent with the data that it is given.
- The ID3 algorithm builds the tree in a recursive, depth-first manner, beginning at the root node and working down to the leaf nodes.

ID3 Algorithm Simplified

- 1 Start by selecting the optimal feature test (i.e., the best question to ask first) for the root node based on **information gain**.
- 2 Label the root node with this feature test.
- 3 Partition the training dataset according to the feature's values.
- 4 Grow branches from the node for each partition created.
- 5 Recursively apply steps 1-4 for each branch, using the partitioned dataset and excluding the previously used features.

Stopping Criteria for ID3 Algorithm

The recursion halts and a leaf node is created in any of these scenarios:

- 1 If all instances in the dataset share the same class, return a leaf node labeled with this class.
- 2 If there are no remaining features to test, return a leaf node labeled with the majority class.
- 3 If the dataset is empty, return a leaf node labeled with the majority class from the parent node.

Predicting Vegetation Distributions

Table: The vegetation classification dataset.

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	high	chaparral
2	true	moderate	low	riparian
3	true	steep	medium	riparian
4	false	steep	medium	chaparral
5	false	flat	high	conifer
6	true	steep	highest	conifer
7	true	steep	high	chaparral

Predicting Vegetation Distributions

$$H(\text{VEGETATION}, \mathcal{D})$$

$$= - \sum_{I \in \left\{ \begin{array}{l} \text{'chaparral'}, \\ \text{'riparian'}, \\ \text{'conifer'} \end{array} \right\}} P(\text{VEGETATION} = I) \times \log_2 (P(\text{VEGETATION} = I))$$

$$= - \left(\left(\frac{3}{7} \times \log_2 \left(\frac{3}{7} \right) \right) + \left(\frac{2}{7} \times \log_2 \left(\frac{2}{7} \right) \right) + \left(\frac{2}{7} \times \log_2 \left(\frac{2}{7} \right) \right) \right)$$

$$= 1.5567 \text{ bits}$$

Predicting Vegetation Distributions

Table: Partition sets (Part.), entropy, remainder (Rem.), and information gain (Info. Gain) by feature for the dataset in Table 2 ^[46].

Split By Feature	Level	Part.	Instances	Partition Entropy	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_1$	$\mathbf{d}_2, \mathbf{d}_3, \mathbf{d}_6, \mathbf{d}_7$	1.5	1.2507	0.3060
	'false'	$\mathcal{D}_2$	$\mathbf{d}_1, \mathbf{d}_4, \mathbf{d}_5$	0.9183		
SLOPE	'flat'	$\mathcal{D}_3$	$\mathbf{d}_5$	0	0.9793	0.5774
	'moderate'	$\mathcal{D}_4$	$\mathbf{d}_2$	0		
	'steep'	$\mathcal{D}_5$	$\mathbf{d}_1, \mathbf{d}_3, \mathbf{d}_4, \mathbf{d}_6, \mathbf{d}_7$	1.3710		
ELEVATION	'low'	$\mathcal{D}_6$	$\mathbf{d}_2$	0	0.6793	0.8774
	'medium'	$\mathcal{D}_7$	$\mathbf{d}_3, \mathbf{d}_4$	1.0		
	'high'	$\mathcal{D}_8$	$\mathbf{d}_1, \mathbf{d}_5, \mathbf{d}_7$	0.9183		
	'highest'	$\mathcal{D}_9$	$\mathbf{d}_6$	0		

Predicting Vegetation Distributions

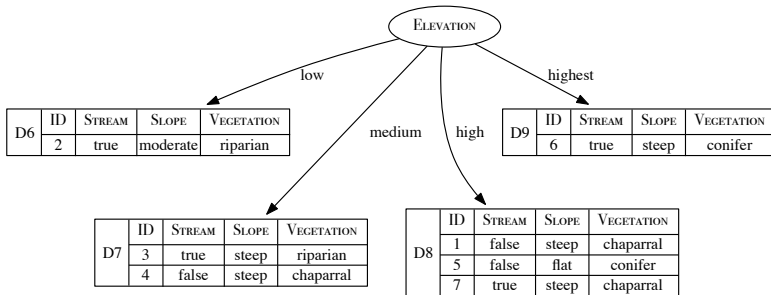


Figure: The decision tree after the data has been split using ELEVATION.

Predicting Vegetation Distributions

$$H(\text{VEGETATION}, \mathcal{D}_7)$$

$$= - \sum_{I \in \left\{ \begin{array}{l} \text{'chaparral'}, \\ \text{'riparian'}, \\ \text{'conifer'} \end{array} \right\}} P(\text{VEGETATION} = I) \times \log_2(P(\text{VEGETATION} = I))$$

$$= - \left(\left(\frac{1}{2} \times \log_2\left(\frac{1}{2}\right) \right) + \left(\frac{1}{2} \times \log_2\left(\frac{1}{2}\right) \right) + \left(\frac{0}{2} \times \log_2\left(\frac{0}{2}\right) \right) \right)$$

$$= 1.0 \text{ bits}$$

Predicting Vegetation Distributions

Table: Partition sets (Part.), entropy, remainder (Rem.) and information gain (Info. Gain) by feature for the dataset $\mathcal{D}_7$ in Figure 8 [49].

Split By Feature	Level	Part.	Instances	Partition Entropy	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_{10}$	$\mathbf{d}_3$	0	0	1.0
	'false'	$\mathcal{D}_{11}$	$\mathbf{d}_4$	0		
SLOPE	'flat'	$\mathcal{D}_{12}$	$\mathbf{d}_3, \mathbf{d}_4$	0	1.0	0
	'moderate'	$\mathcal{D}_{13}$		0		
	'steep'	$\mathcal{D}_{14}$		1.0		

A Worked Example: Predicting Vegetation Distributions

Predicting Vegetation Distributions

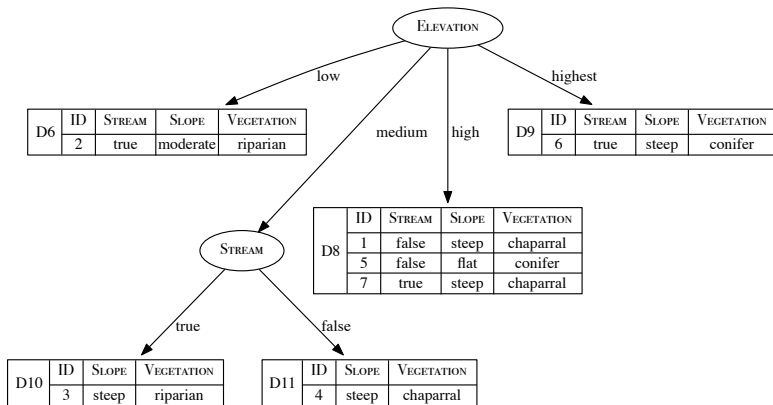


Figure: The state of the decision tree after the $\mathcal{D}_7$ partition has been split using STREAM.

Predicting Vegetation Distributions

$$H(\text{VEGETATION}, \mathcal{D}_8)$$

$$= - \sum_{I \in \left\{ \begin{array}{l} \text{'chaparral'}, \\ \text{'riparian'}, \\ \text{'conifer'} \end{array} \right\}} P(\text{VEGETATION} = I) \times \log_2(P(\text{VEGETATION} = I))$$

$$= - \left(\left(\frac{2}{3} \times \log_2\left(\frac{2}{3}\right) \right) + \left(\frac{0}{3} \times \log_2\left(\frac{0}{3}\right) \right) + \left(\frac{1}{3} \times \log_2\left(\frac{1}{3}\right) \right) \right)$$

$$= 0.9183 \text{ bits}$$

Predicting Vegetation Distributions

Table: Partition sets (Part.), entropy, remainder (Rem.) and information gain (Info. Gain) by feature for the dataset $\mathcal{D}_8$ in Figure 9 [52].

Split By Feature	Level	Part.	Instances	Partition Entropy	Rem.	Info. Gain
STREAM	'true'	$\mathcal{D}_{15}$	$\mathbf{d}_7$	0	0.6666	0.2517
	'false'	$\mathcal{D}_{16}$	$\mathbf{d}_1, \mathbf{d}_5$	1.0		
SLOPE	'flat'	$\mathcal{D}_{17}$	$\mathbf{d}_5$	0	0	0.9183
	'moderate'	$\mathcal{D}_{18}$		0		
	'steep'	$\mathcal{D}_{19}$	$\mathbf{d}_1, \mathbf{d}_7$	0		

A Worked Example: Predicting Vegetation Distributions

Predicting Vegetation Distributions

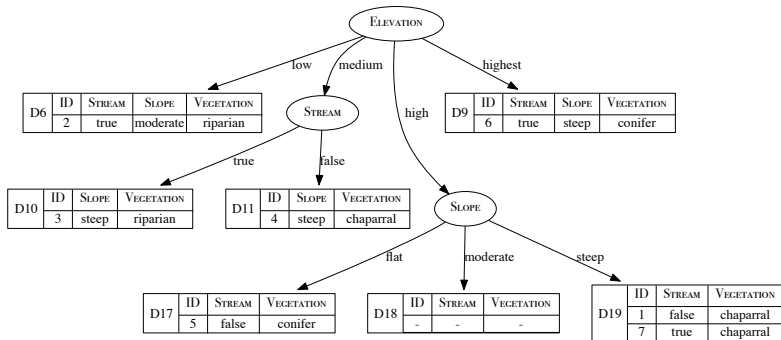


Figure: The state of the decision tree after the $\mathcal{D}_8$ partition has been split using SLOPE.

Predicting Vegetation Distributions

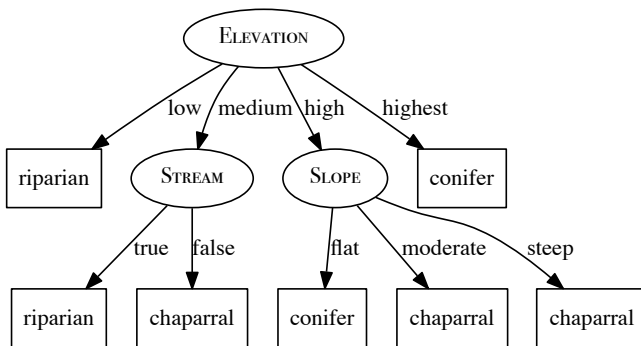
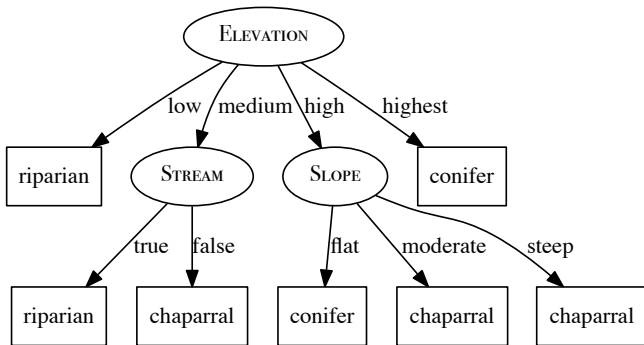


Figure: The final vegetation classification decision tree.

Predicting Vegetation Distributions

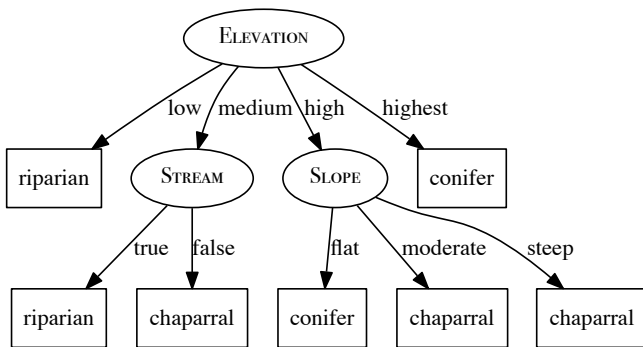


- What prediction will this decision tree model return for the following query?

STREAM = *'true'*, SLOPE=*'Moderate'*, ELEVATION=*'High'*

A Worked Example: Predicting Vegetation Distributions

Predicting Vegetation Distributions



- What prediction will this decision tree model return for the following query?

STREAM = 'true', SLOPE='Moderate', ELEVATION='High'

VEGETATION = 'Chaparral'

Predicting Vegetation Distributions

ID	STREAM	SLOPE	ELEVATION	VEGETATION
1	false	steep	high	chaparral
2	true	moderate	low	riparian
3	true	steep	medium	riparian
4	false	steep	medium	chaparral
5	false	flat	high	conifer
6	true	steep	highest	conifer
7	true	steep	high	chaparral

STREAM = *'true'*, SLOPE = *'Moderate'*, ELEVATION = *'High'*

VEGETATION = *'Chaparral'*

- This is an example where the model is attempting to **generalize** beyond the dataset.
- Whether or not the generalization is correct depends on whether the assumptions used in generating the model (i.e. the **inductive bias**) were appropriate.

Alternative Decision Tree Algorithms: C4.5 and CART

C4.5 Overview:

- Utilizes **gain ratio** to select the optimal feature, improving upon ID3's information gain.
- Introduces pruning to handle overfitting and noise.

CART (Classification and Regression Trees):

- Can be used for classification and regression.
- Chooses the best feature using the **Gini index** for classification, or least squares deviation for regression.
- Use only binary splits.

Stopping Criteria:

- In addition to ID3 stop conditions, C4.5 and CART use extra conditions like reaching a maximum depth.
- CART specifically may stop when further splitting does not significantly reduce impurity or when a minimum subset size is reached.

Summary

Summary

- The ID3 algorithm works the same way for larger more complicated datasets.
- There have been many extensions and variations proposed for the ID3 algorithm:
 - using different impurity measures (Gini, Gain Ratio)
 - handling continuous descriptive features
 - to handle continuous targets
 - pruning to guard against over-fitting
 - using decision trees as part of an ensemble (Random Forests)
- These extensions are covered in Section 4.4.

1 Big Idea

2 Fundamentals

- Decision Trees
- Shannon's Entropy Model
- Information Gain

3 Standard Approach: The ID3 Algorithm

- A Worked Example: Predicting Vegetation Distributions
- Alternative Decision Tree Algorithms: C4.5 and CART

4 Summary

Slide Acknowledgment

The slides used in this course are based on the official textbook materials, with modifications made where necessary to suit the course requirements and enhance the learning experience.